Innovation Centre for Education

NCS NihonCommunicationSolutions

Yenepoya Institute of Arts, Science, Commerce and Management

Project – Low Level Design

on

Multi-Cloud Hybrid Strategy for Healthcare

Student Name: Adharv Shyam

Industry Mentor:

Mr. Sumit Kumar Shukla

Guided by:

Mr. Praveen

Table of Contents

1. Introduction.....3
-1.1 Purpose.....3
-1.2 Scope.....3
2. System Overview.....3
-2.1 Main Workflow.....3
-2.2 Modules Used.....4
3. Detailed Module Design.....4
-3.1 main.py.....4
-3.2 backup_engine.py.....5
-3.3 file_manager.py.....5
-3.4 cloud_uploader.py.....5
-3.5 s3_uploader.py.....5
-3.6 logger.py and dashboard.py.....6
4. Data Design.....6
-4.1 Source Files.....6
-4.2 Backup Storage.....6
5. Process Flow.....7
-5.1 Local Backup Flow.....7
-5.2 Cloud Backup Flow.....7
-5.3 Failover Flow.....7
6. Security and Limits.....7
7**. Future Improvements**.....8
8. Conclusion.....8

1. Introduction

This document explains the low level design of the Multi-Cloud Hybrid Strategy for Healthcare project. It shows how the code works, how files move through the system, and how local and cloud backups are handled.

1.1 Purpose

The purpose of this LLD is to explain the internal working of the project in a simple and clear way. It covers the modules, functions, data files, backup steps, and failover logic used in the implementation.

1.2 Scope

This document covers the Python scripts used in the project, the healthcare CSV dataset, the local backup folder, Google Drive upload, AWS S3 upload, logging, and failover handling.

2. System Overview

The project reads healthcare CSV files from the data folder, creates a copy in the local backup folder, checks integrity, and then uploads the files to Google Drive and AWS S3. A scheduler runs this process automatically.

2.1 Main Workflow

- Read healthcare files from the dataset folder.
- Copy each file into the local backup folder.
- Check SHA256 hash and file size.
- Upload the file to Google Drive.
- Upload the file to AWS S3.
- Store logs and show backup status.
- If cloud upload fails, keep the local backup as failover.

2.2 Modules Used

Module	Use
main.py	Starts the scheduler and triggers the backup run.
backup_engine.py	Controls the full backup process.
file_manager.py	Finds files and calculates hashes.
cloud_uploader.py	Handles Google Drive upload.
s3_uploader.py	Handles AWS S3 upload.
logger.py	Writes backup logs.
dashboard.py	Shows latest backup events.
report_manager.py	Stores backup counters and summary.

3. Detailed Module Design

3.1 main.py

This file is the entry point of the project. It schedules the backup job every 10 minutes, runs one backup immediately, and keeps the script active.

- Imports schedule, time, datetime, perform_backup, and show_dashboard.
- Defines run_backup() as the main execution function.
- Calls perform_backup() and then show_dashboard().
- Uses an infinite loop to keep checking scheduled jobs.

3.2 backup_engine.py

This is the main logic file of the project. It handles local copy, verification, cloud upload, failover, and report update.

- Gets all files from data/healthcare_data.
- Creates backup/local_backup if it does not exist.
- Copies each file with shutil.copy().
- Checks hash and file size after copying.
- Calls Google Drive and AWS S3 upload functions.
- Marks failover if both cloud uploads fail.
- Prints the final backup summary.

3.3 file_manager.py

This file supports the backup engine by providing file utility functions.

- get_all_files(folder_path) scans all files using os.walk().
- get_file_hash(file_path) creates SHA256 hash using binary read mode.
- Hash is used to compare original and copied files.

3.4 cloud_uploader.py

This module uploads files to Google Drive. It also checks whether the backup folder already exists and updates files when needed.

- authenticate_drive() loads or refreshes Google credentials.
- get_backup_folder(service) finds or creates the Healthcare_Backups folder.
- upload_file(file_path) uploads a new file or updates an existing one.

3.5 s3_uploader.py

This module uploads files to AWS S3 using boto3 and values from aws_config.json.

- load_aws_config() reads access key, secret key, region, and bucket name.
- upload_to_s3(file_path) uploads the file to the target bucket.
- If upload fails, the error is returned to the backup engine.

3.6 logger.py and dashboard.py

These files handle monitoring output of the project.

- logger.py stores timestamped messages in logs/backup_log.txt.
- dashboard.py reads the log file and prints the latest 5 events.
- This helps the user quickly check what happened in the last run.

4. Data Design

The project uses CSV files as the data source. It does not use a database. The backup system works directly with files.

4.1 Source Files

File	Main Fields
patients.csv	patient_id, name, age, gender, blood_group, disease, doctor, city
appointments.csv	appointment_id, patient_id, doctor, date, time, status
lab_reports.csv	report_id, patient_id, test_name, result, date
prescriptions.csv	prescription_id, patient_id, medicine, dosage, doctor

4.2 Backup Storage

- Original data is stored in data/healthcare_data.
- Local copies are stored in backup/local_backup.
- Google Drive stores backup files in the Healthcare_Backups folder.
- AWS S3 stores backup files inside the configured bucket.
- Logs are stored in logs/backup_log.txt.

5. Process Flow

5.1 Local Backup Flow

- Start backup run.
- Read all dataset files.
- Copy each file to the local backup folder.
- Verify the copied file using SHA256 and size check.

5.2 Cloud Backup Flow

- Send file to Google Drive.
- Send file to AWS S3.
- Update report counters and log success or failure.

5.3 Failover Flow

- If Google Drive fails, the process still tries AWS S3.
- If AWS S3 fails, the process still keeps the local copy.
- If both cloud uploads fail, failover is marked in the report.
- Because of this, backup data is not lost.

6. Security and Limits

This project is mainly for learning and demonstration. It uses file-based credentials and does not yet include encryption or access control. So it explains the backup design well, but it would need stronger security for real healthcare use.

- Credentials are stored in config files.
- Uploaded files are not encrypted by custom project logic.
- There is no user login system.
- Logs may grow over time without rotation.

7. Future Improvements

- Add encryption before cloud upload.
- Use environment variables for secrets.
- Add alerts on backup failure.
- Support more cloud providers.
- Create a simple web dashboard.

8. Conclusion

The project has a simple and useful low level design. It shows how healthcare files can be backed up to local storage and multiple cloud platforms with verification, logging, and failover support.